Week 4: PostgreSQL Core — Data Types, Constraints, and PG-Specific Features

Phase 2: PostgreSQL Mastery | Week 4 of 12

Week Overview

This week dives into PostgreSQL-specific features that don't exist in standard SQL. You will master the rich type system (arrays, JSONB, UUID, range types), learn to enforce data integrity beyond basic constraints, write your first PL/pgSQL functions, and understand how sequences and identity columns work under the hood.

Focus: JSONB and PL/pgSQL are the two most differentiating PostgreSQL skills. Invest heavily here.

Learning Objectives

By the end of this week, you will be able to:

- Use PostgreSQL's extended data types: ARRAY, JSONB, UUID, INET, CIDR, RANGE.
- Write table-level and column-level constraints with CHECK, UNIQUE, and EXCLUSION.
- Create custom DOMAIN types for reusable validation.
- Write basic PL/pgSQL functions and procedures.
- Use sequences and IDENTITY columns correctly.
- Understand GENERATED ALWAYS AS computed columns.
- Perform JSONB operations: extraction, path queries, containment, GIN indexing.
- Use array operators: @>, ANY, ALL, unnest, array_agg.

Required Reading

- `05_PostgreSQL_Core/` — All files

Daily Schedule

Monday — Extended Data Types (60 min)

Topics:

- NUMERIC precision vs. FLOAT
- TEXT vs. VARCHAR — when it matters
- UUID for distributed IDs
- ARRAY type and operators
- INET and CIDR for IP addresses
- RANGE types: daterange, tsrange, numrange

```
-- Arrays
CREATE TABLE posts (
    id      SERIAL PRIMARY KEY,
    title   VARCHAR(300),
    tags    TEXT[],
```

```
    scores  INTEGER[]
);

INSERT INTO posts (title, tags, scores) VALUES
  ('PostgreSQL Arrays', ARRAY['postgres','sql','database'], ARRAY[95, 88, 92]),
  ('Python Tutorial',   ARRAY['python','programming'],       ARRAY[80, 75]);

-- Array operators
SELECT * FROM posts WHERE tags @> ARRAY['sql'];           -- contains
SELECT * FROM posts WHERE 'python' = ANY(tags);           -- any match
SELECT title, tags[1] AS first_tag FROM posts;             -- subscript access
SELECT unnest(tags) AS tag FROM posts;                     -- expand to rows
SELECT title, array_length(tags, 1) AS tag_count FROM posts;

-- RANGE types
CREATE TABLE room_bookings (
    room_id  INTEGER,
    during   TSRANGE NOT NULL,
    EXCLUDE USING GIST (room_id WITH =, during WITH &&)  -- no overlaps
);

INSERT INTO room_bookings VALUES (1, '[2024-12-01 09:00, 2024-12-01 10:00)');
INSERT INTO room_bookings VALUES (1, '[2024-12-01 09:30, 2024-12-01 11:00)');
-- Second INSERT should fail due to EXCLUDE constraint

-- UUID
SELECT gen_random_uuid();
CREATE EXTENSION IF NOT EXISTS pgcrypto;
CREATE TABLE events_log (
    id         UUID DEFAULT gen_random_uuid() PRIMARY KEY,
    event_name VARCHAR(200),
    occurred_at TIMESTAMPTZ DEFAULT NOW()
);
```

Tuesday — JSONB Deep Dive (90 min)

Topics:

- JSON vs. JSONB (JSONB is parsed, indexed, stored compressed)
- Extraction operators: `->`, `->>`, `#>`, `#>>`
- Containment: `@>`, `<@`
- Key existence: `?`, `?|`, `?&`
- Modifying JSONB: `||` (merge), `jsonb_set`, `jsonb_strip_nulls`
- Building JSONB: `jsonb_build_object`, `to_jsonb`, `row_to_json`
- GIN indexing for JSONB

```
CREATE TABLE products (
    id          SERIAL PRIMARY KEY,
    name        VARCHAR(200),
    attributes  JSONB NOT NULL DEFAULT '{}'
);
```

```sql
INSERT INTO products (name, attributes) VALUES
  ('Laptop Pro',
'{"brand":"Apple","ram_gb":16,"storage_gb":512,"color":"Silver","tags":
["portable","premium"]}'),
  ('Phone X',
'{"brand":"Samsung","ram_gb":8,"storage_gb":256,"color":"Black","tags":
["smartphone"]}'),
  ('Tablet Air', '{"brand":"Apple","ram_gb":8,"storage_gb":64,"color":"Gold","tags":
["portable"]}');

-- Extraction
SELECT name, attributes->'brand' AS brand_json,     -- returns JSON "Apple"
             attributes->>'brand' AS brand_text      -- returns text Apple
FROM products;

-- Nested path
SELECT name, attributes#>>'{tags,0}' AS first_tag FROM products;

-- Containment queries
SELECT * FROM products WHERE attributes @> '{"brand":"Apple"}';
SELECT * FROM products WHERE attributes @> '{"tags":["portable"]}';

-- Key existence
SELECT * FROM products WHERE attributes ? 'color';
SELECT * FROM products WHERE attributes ?| ARRAY['ram_gb','storage_gb'];

-- Modifying
UPDATE products
SET attributes = jsonb_set(attributes, '{color}', '"Space Gray"')
WHERE name = 'Laptop Pro';

UPDATE products
SET attributes = attributes || '{"price": 1299.99}'::JSONB
WHERE name = 'Laptop Pro';

-- GIN index for fast JSONB queries
CREATE INDEX idx_products_attrs ON products USING GIN (attributes);

-- Expand JSONB to rows
SELECT name, key, value
FROM products, jsonb_each_text(attributes);
```

Wednesday — Constraints and Domains (90 min)

Topics:

- CHECK constraints (column and table-level)
- UNIQUE constraints and UNIQUE partial indexes
- EXCLUSION constraints (range overlap prevention)
- DEFERRABLE constraints

- Custom DOMAIN types
- GENERATED ALWAYS AS computed columns

```
-- Custom domain for validation
CREATE DOMAIN email_address AS VARCHAR(300)
    CHECK (VALUE ~* '^[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}$');

CREATE DOMAIN positive_amount AS NUMERIC(12,2)
    CHECK (VALUE > 0);

-- Use domain in table
CREATE TABLE invoices (
    invoice_id  SERIAL PRIMARY KEY,
    email       email_address NOT NULL,
    amount      positive_amount NOT NULL,
    tax_rate    NUMERIC(5,4) DEFAULT 0.09 CHECK (tax_rate BETWEEN 0 AND 1),
    -- Computed column
    tax_amount  NUMERIC(12,2) GENERATED ALWAYS AS (ROUND(amount * tax_rate, 2))
STORED,
    total       NUMERIC(12,2) GENERATED ALWAYS AS (ROUND(amount * (1 + tax_rate),
2)) STORED,
    status      VARCHAR(20) DEFAULT 'Draft'
                CHECK (status IN ('Draft','Sent','Paid','Overdue','Cancelled'))
);

INSERT INTO invoices (email, amount) VALUES
  ('client@example.com', 1000.00),
  ('team@company.org',   2500.00);

-- Verify computed columns
SELECT invoice_id, amount, tax_amount, total FROM invoices;

-- DEFERRABLE constraints (batch processing pattern)
CREATE TABLE accounts (id SERIAL PRIMARY KEY, balance NUMERIC);
CREATE TABLE transfers (
    from_id  INTEGER REFERENCES accounts(id) DEFERRABLE INITIALLY DEFERRED,
    to_id    INTEGER REFERENCES accounts(id) DEFERRABLE INITIALLY DEFERRED,
    amount   NUMERIC
);
```

Thursday — PL/pgSQL Fundamentals (60 min)

Topics:

- Function vs. Procedure syntax
- DECLARE, variable assignment
- IF / ELSIF / ELSE
- FOR loops and WHILE loops
- Exception handling with EXCEPTION and RAISE
- RETURN TABLE for set-returning functions

```sql
-- Basic function
CREATE OR REPLACE FUNCTION calculate_tax(amount NUMERIC, tax_rate NUMERIC DEFAULT
0.09)
RETURNS NUMERIC AS $$
BEGIN
    IF amount < 0 THEN
        RAISE EXCEPTION 'Amount cannot be negative: %', amount;
    END IF;
    RETURN ROUND(amount * tax_rate, 2);
END;
$$ LANGUAGE plpgsql;

SELECT calculate_tax(1000), calculate_tax(500, 0.20);

-- Function returning a table
CREATE OR REPLACE FUNCTION get_department_stats(p_dept VARCHAR)
RETURNS TABLE (metric TEXT, value NUMERIC) AS $$
BEGIN
    RETURN QUERY
    SELECT 'headcount'::TEXT, COUNT(*)::NUMERIC FROM employees WHERE department =
p_dept;

    RETURN QUERY
    SELECT 'avg_salary'::TEXT, ROUND(AVG(salary), 2) FROM employees WHERE department
= p_dept;

    RETURN QUERY
    SELECT 'max_salary'::TEXT, MAX(salary) FROM employees WHERE department = p_dept;
END;
$$ LANGUAGE plpgsql;

SELECT * FROM get_department_stats('Engineering');

-- Procedure with transaction control
CREATE OR REPLACE PROCEDURE transfer_salary(
    from_emp INTEGER,
    to_emp   INTEGER,
    amount   NUMERIC
)
LANGUAGE plpgsql AS $$
BEGIN
    UPDATE employees SET salary = salary - amount WHERE id = from_emp;
    UPDATE employees SET salary = salary + amount WHERE id = to_emp;
    -- COMMIT is implicit in procedure called without outer transaction
END;
$$;

CALL transfer_salary(1, 2, 5000);
```

Friday — Review + JSONB API Mini-Project (45 min)

Mini-Project: Build a flexible product catalog using JSONB.

```
-- Schema: products with flexible JSONB attributes
-- Challenge: support electronics (RAM, storage), clothing (size, material), food
(calories, allergens)
-- Write queries:
-- 1. Filter electronics by RAM >= 16GB
-- 2. Find all products with 'portable' tag
-- 3. Count products by brand
-- 4. Average price by category
-- 5. Find products missing a required attribute
```

Practice Tasks

1. Create a `hotel_rooms` table with an EXCLUSION constraint to prevent overlapping bookings.
2. Write a PL/pgSQL function that generates a random 8-character alphanumeric password.
3. Store user preferences as JSONB and write a query to find users with `theme = 'dark'`.
4. Use `unnest()` to convert a JSONB array into individual rows.
5. Write a function that validates a credit card number using the Luhn algorithm.
6. Create a DOMAIN for `percentage` (0-100) and use it in a grades table.
7. Use `jsonb_agg` to build a JSONB array of all tags per product category.
8. Write a computed column that calculates `discount_price` as 90% of `price`.

Self-Assessment Checklist

- [ ] I can use JSONB containment (`@>`) and extraction (`->>`) operators
- [ ] I created a GIN index on a JSONB column and verified it's used
- [ ] I wrote a custom DOMAIN with a CHECK constraint
- [ ] I wrote a PL/pgSQL function with IF/ELSE and exception handling
- [ ] I understand GENERATED ALWAYS AS computed columns
- [ ] I used arrays with `ANY` and `@>` operators
- [ ] I understand when JSONB is appropriate vs. a normalized schema

---

Mock Interview Questions

1. When would you use JSONB instead of a normalized relational schema?
2. What is the performance difference between JSON and JSONB in PostgreSQL?
3. How do you index JSONB data for fast containment queries?
4. What is the difference between `->` and `->>` in JSONB extraction?
5. Explain a use case for a DOMAIN type.
6. What is a GENERATED ALWAYS AS column? When is it useful?
7. How do EXCLUSION constraints differ from UNIQUE constraints?
8. Write a PL/pgSQL function that raises an exception if a balance goes negative.

Resources

- This repo: `05_PostgreSQL_Core/`

- JSONB docs: https://www.postgresql.org/docs/16/datatype-json.html
- PL/pgSQL reference: https://www.postgresql.org/docs/16/plpgsql.html